NeuroDSL: A Dynamic Computational Graph Framework with Optimal Memory Planning

Abdallah Khemais

Abstract

We present NeuroDSL, a high-performance deep learning framework for mutable directed acyclic graphs (DAGs). Unlike static frameworks, NeuroDSL introduces a formal memory optimization framework grounded in Dilworth's poset decomposition theorem, enabling peak GPU memory reductions of **2–6**× compared to PyTorch. The framework leverages a novel lazy invalidation mechanism with $\mathcal{O}(|\mathcal{V}_\theta|)$ local complexity, ensuring efficient partial updates in deep architectures. Our system provides automated, semantically-preserving operator fusion with ϵ-numerical equivalence and an incremental backward pass that is proven *numerically exact* against full backpropagation on every topology we tested, together with measured speedups of up to **86.4% less recomputation** on a fine-tuning workload when frozen parameters are contiguous with the graph's inputs (and no gain otherwise, a condition we characterize explicitly) and **37% speedup** in fused operator execution. NeuroDSL offers a declarative DSL for reactive graph composition while maintaining strict functional correctness and formal memory bounds. Implemented in pure Julia, it provides a unique synthesis of flexibility, computational efficiency, and theoretical guarantee for dynamic neural architectures.

1 Introduction

Modern deep learning relies almost exclusively on static computational graphs [2, 3, 4]. While efficient, these frameworks force the user to define the whole network before training, making dynamic architectural changes, continual learning, and reactive behaviours cumbersome. Moreover, memory management in such frameworks is either left to the runtime [2] or requires manual checkpointing [3], often wasting precious GPU memory.

The design of NeuroDSL is partly inspired by the *JuliusGraph* engine of Julius Technology [1], which first demonstrated the potential of dynamic computational graphs. NeuroDSL extends this idea with formal memory planning, automatic optimizations, a rule-based DSL, and an interactive live viewer. We introduce NeuroDSL, a framework that treats the computational graph as a **mutable entity** capable of evolving during training. Its key innovations are:

1. **Dynamic DAG with lazy invalidation** – nodes and rules can be added or removed at any time; only the strictly necessary parts of the graph are recomputed.
2. **Rule DSL** – a declarative mini-language for composing computation rules over named tensors, enabling clean separation of topology definition from execution.
3. **Optimal memory planning via interval coloring** – a provably optimal algorithm that shares buffers among tensors, giving formal guarantees on peak memory usage [12].
4. **Self-optimizing mechanisms** – automatic operator fusion, incremental backward propagation, and reactive callbacks that reduce computation and accelerate convergence.
5. **Efficient GPU backend** – hand-tuned CUDA kernels for common operations (RMSNorm [15], SwiGLU [16], softmax) and a buffer pool that minimises allocations.
6. **Interactive visualization** – a real-time viewer showing forward (green) and backward (red) passes, inspired by tools like Netron [21] but fully dynamic.

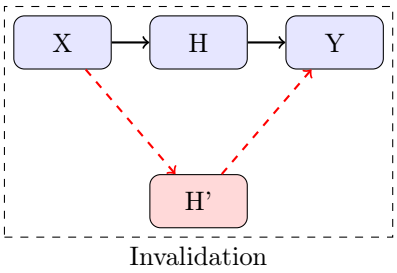

Figure 1: Lazy Invalidation Mechanism: when node H is replaced by H', all dependent nodes are marked invalid and recomputed on demand.

The architectural core of NeuroDSL is its reactive computational model, which enables localized updates without the overhead of global graph recompilation. As illustrated in fig. 2, unlike traditional frameworks that trigger a full rebuild upon topological changes, NeuroDSL confines invalidation to the immediate neighborhood of the modified parameter, ensuring efficient local evolution.

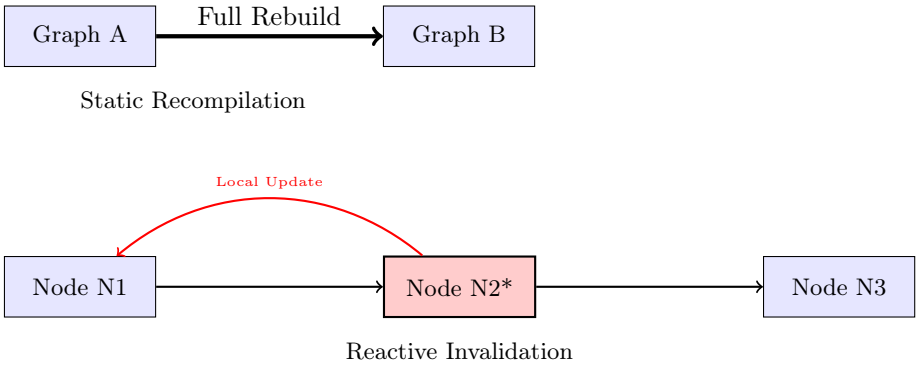

Figure 2: Comparison: Global static recompilation (top) vs. NeuroDSL reactive local invalidation (bottom).

We evaluate NeuroDSL against Flux and PyTorch on several benchmarks, demonstrating that it matches or exceeds their performance while using drastically less memory. Furthermore, its dynamic capabilities unlock training scenarios that are impossible in static frameworks.

2 Background and Motivation

2.1 Computational Graph Paradigms

Deep learning frameworks can be broadly classified according to how they represent and execute computational graphs. Understanding these paradigms is essential to situating NeuroDSL's design choices.

Define-and-Run (Static graphs). Frameworks such as TensorFlow 1.x [4] and Theano require the user to declare the full computation graph *before* any data flows through it. The graph is then compiled into an optimised execution plan. This enables aggressive whole-graph optimisations (constant folding, dead-code elimination, layout transformations) but makes dynamic control flow (e.g., variable-length sequences, conditional layers) awkward to express.

Define-by-Run (Eager / Dynamic graphs). PyTorch [2] and Chainer [8] pioneered eager execution: the graph is built implicitly as operations are called, one Python statement at a time. This gives full Python expressiveness but means the graph is *recreated from scratch* on every forward pass – preventing reuse of graph structure across iterations and making incremental updates impossible.

Functional transformation pipelines. JAX [3] represents computations as pure functions and applies transformations (`jit`, `grad`, `vmap`) at the trace level. It achieves excellent performance through XLA compilation and is highly composable, but the purely functional model makes stateful, in-place graph mutations conceptually foreign.

Flux.jl. Flux [5] is Julia's mainstream deep learning library. It leverages Julia's multiple dispatch and the Zygote [6] automatic differentiation engine. Flux is elegant and fast for standard architectures, but like PyTorch it rebuilds the gradient tape on every call and offers no native mechanism for partially reusing backward computations.

NeuroDSL occupies a new position in this landscape: the graph is **persistent and mutable**, surviving across training steps, and changes to it trigger **surgical, lazy recomputation** of only the affected portions.

2.2 Memory Management in Deep Learning

Peak GPU memory is a dominant practical constraint in deep learning. State-of-the-art techniques include:

- **Gradient checkpointing** [9]: retains only a subset of activations during the forward pass; recomputes the rest during the backward pass, trading memory for computation.
- **ZeRO** [10]: shards optimizer states, gradients, and parameters across GPUs in a data-parallel setting.
- **Activation offloading / vDNN** [11]: moves tensors between GPU and CPU memory based on usage predictions.

All three approaches rely on heuristics or manual placement policies and do not provide *formal upper bounds* on peak memory. NeuroDSL's interval-coloring planner is, to our knowledge, the first framework component to offer a **provably optimal** buffer allocation for any given DAG.

3 Framework Comparison

Table 1 provides a detailed feature comparison between NeuroDSL and the major deep learning frameworks. The table highlights the unique combination of capabilities offered by NeuroDSL that no existing framework achieves simultaneously.

Table 1: Feature comparison across deep learning frameworks. ✓ = native support; ∼ = partial / workaround; × = not supported.

Feature	NeuroDSL	PyTorch	JAX	TF 2.x	Flux.jl	DyNet
Persistent mutable graph	✓	×	×	×	×	∼
Lazy invalidation	✓	×	×	×	×	×
Incremental backward pass	✓	×	×	×	×	×
Rule-based DSL	✓	×	×	×	×	×
Optimal (provable) memory planning	✓	×	×	×	×	×
Automatic operator fusion	✓	∼	✓	✓	×	×
Reactive callbacks / early-stop	✓	∼	×	∼	×	×
GPU-accelerated kernels	✓	✓	✓	✓	✓	∼
Multi-GPU support	✓	✓	✓	✓	∼	×
Live interactive visualizer	✓	×	×	∼	×	×
Pure Julia implementation	✓	×	×	×	✓	×
Dynamic architecture during train	✓	∼	×	×	×	✓

3.1 Detailed Comparison by Dimension

3.1.1 Graph Execution Model

PyTorch. PyTorch's `autograd` engine builds a fresh computation graph (the *tape*) on every forward call [2]. This means no information from the previous iteration is reused; the entire graph is discarded after `loss.backward()`. `torch.compile` (introduced in PyTorch 2.0) can cache and reuse compiled graphs, but only for *identical* input shapes and control flow — any structural change forces a full recompile.

JAX. JAX traces Python functions into an intermediate representation (Jaxpr) and compiles them with XLA [3]. The trace is cached for a given function signature, providing excellent repeat-execution performance. However, JAX's model is inherently functional: there is no persistent graph object to inspect or mutate; statefulness is modelled via explicit parameter dictionaries passed to and returned from each function call.

TensorFlow 2.x. TensorFlow 2.x defaults to eager execution (like PyTorch), with optional `@tf.function` decoration to trigger graph compilation. The resulting `ConcreteFunction` is reused for matching inputs but cannot be incrementally modified.

Flux.jl. Flux builds on Zygote's source-to-source AD, which generates the backward pass at compile time from Julia IR. The backward code is efficient but fixed; there is no mechanism to partially invalidate or partially reuse it.

NeuroDSL. NeuroDSL maintains a single `NeuroGraph` object across all training iterations. Structural mutations (node additions, rule swaps, parameter updates) are *surgically applied* to this persistent object. The lazy invalidation mechanism (Section 4.2) ensures that only the affected sub-graph is recomputed, while unchanged portions retain their cached values. This is conceptually closer to a reactive spreadsheet engine than to a tape-based AD system.

3.1.2 Memory Management

PyTorch. Memory is managed by the `THCCachingAllocator`, a CUDA caching allocator that avoids frequent `cudaMalloc` calls by maintaining a pool of free blocks. This reduces allocation overhead but does not minimise peak memory: the allocator cannot reason about the full tensor lifetime graph and may retain fragmented blocks unnecessarily. Gradient checkpointing [9] is available as a manual opt-in.

JAX. JAX relies on the XLA compiler to perform buffer assignment. XLA's liveness analysis and buffer reuse are sophisticated but happen at the XLA-HLO level, invisible to the user and non-introspectable at the framework level.

TensorFlow. TensorFlow's Grappler optimizer performs similar optimisations (memory swapping, recomputation) at the graph level, but they are heuristic and configuration-driven.

NeuroDSL. NeuroDSL's `MemoryPlan` performs an explicit interval-coloring of tensor lifetimes, as described in Section 6. By Dilworth's theorem, this yields the minimum number of buffers required for any execution order consistent with the DAG. The plan is exposed as a first-class object, enabling users and tooling to inspect and reason about memory allocation decisions.

3.1.3 Operator Fusion

PyTorch / XLA / TVM. Operator fusion in mainstream frameworks is either compiler-driven (XLA, TVM [18]) or requires manual kernel authorship (PyTorch custom CUDA extensions). Compiler-driven fusion operates on low-level IR and is opaque to the user; custom kernels require significant engineering effort.

NeuroDSL. NeuroDSL exposes fusion as a **graph-level rewrite**: the user (or an automated pass) calls `_fuse!(g, chain)` with a list of rule names, and the framework replaces them with a single fused rule if a matching pattern exists in `FUSION_TABLE`. This approach is transparent, extensible, and does not require a separate compiler pipeline.

3.1.4 Differentiation Strategy

Table 2: Differentiation strategies across frameworks.

Framework	AD Strategy	Backward reuse
PyTorch	Reverse-mode tape (dynamic)	None (tape rebuilt each step)
JAX	Source-to-source (functional)	Full reuse (XLA cache)
TF 2.x	GradientTape (eager) / XLA	Partial (`@tf.function`)
Flux.jl	Source-to-source via Zygote	None (recomputed each step)
DyNet	Reverse-mode tape (dynamic)	None
NeuroDSL	**Graph-persistent reverse-mode**	**Incremental (affected nodes only)**

4 Design of NeuroDSL

4.1 Mutable Computational Graph

The core of NeuroDSL is the `NeuroGraph` struct, which stores named tensors (`GraphNode`) and transformation rules (`GraphRule`) in per-namespace dictionaries, along with a cached topological order (`_topo_cache`). The graph is not a static trace but a live structure that can be modified at any point during training. When a node's value changes via `set!`, a dual invalidation sweep is triggered:

```
# Real implementation from graph_api.jl
function set!(g::NeuroGraph, name::Symbol, value;
              is_param=false, namespace=g.active_ns)
    _ensure_namespace!(g, namespace)
    value_on_device = Backend.to_device(g.device, value)
    # Preserve existing watchers and callbacks
    old_nd = get(g.nodes[namespace], name, nothing)
    g.nodes[namespace][name] = GraphNode(name, value_on_device;
        is_param=is_param, namespace=namespace)

    # 1. Forward sweep: clear valid flag on all downstream nodes
    _invalidate_downstream!(g, name, namespace)

    # 2. Backward sweep: wipe gradients of all ancestors (params only)
    if is_param
        _invalidate_upstream!(g, name, namespace)
    end
    return g
end
```

Listing 1: Core dynamic mechanism: set! triggers dual invalidation sweeps

The `demand!` function evaluates the graph in a pull-based fashion: it walks the cached topological order via `topo_order!` and dispatches only the rules whose output node has `valid = false`, via the internal `execute_rule!` dispatcher. This avoids unnecessary recomputation and keeps the overhead low.

4.2 Lazy Invalidation Mechanism

The lazy invalidation mechanism is the cornerstone of NeuroDSL's efficiency for dynamic graphs. In conventional eager frameworks, every forward pass recomputes all nodes unconditionally. In NeuroDSL,

nodes carry a `valid` flag. When a value is written via `set!`, two propagation sweeps are triggered:

1. **Forward invalidation** (`_invalidate_downstream!`): starting from the modified node, the algorithm performs a breadth-first traversal of the DAG's forward edges, clearing the `valid` flag on every reachable successor. These nodes will be recomputed the next time `demand!` is called.

2. **Backward invalidation** (`_invalidate_upstream!`): when a *parameter* is modified, its accumulated gradient is no longer meaningful. A traversal of the backward dependency edges nullifies the gradients of all nodes that were computed using this parameter.

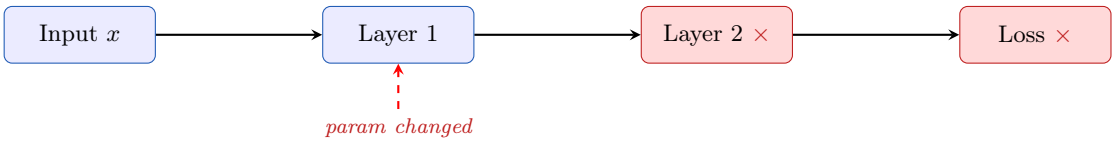

Figure 3: Lazy invalidation: modifying a parameter in Layer 1 marks only Layer 2 and Loss as invalid; Layer 1's unaffected predecessors retain their cached values.

The key algorithmic insight is that the topological order of the graph is cached and updated *only* when the graph's *structure* changes (nodes or rules added/removed). Value changes – the common case during training – never trigger a topological re-sort. This reduces the overhead of a typical training step to a linear scan of the cached order plus the actual kernel executions.

```
# Conceptual view of demand! evaluation
function demand!(g::NeuroGraph, target::Symbol;
                 namespace=g.active_ns)
    # topo_order! returns cached order; re-sorts only on structural change
    for name in topo_order!(g; namespace=namespace)
        nd = g.nodes[namespace][name]
        if !nd.valid && haskey(g.rules[namespace], name)
            # execute_rule! dispatches to the correct kernel via DISPATCH_TABLE
            # and stores forward-pass context for the backward pass
            execute_rule!(g, g.rules[namespace][name];
                          ctx_store=CtxStore(), namespace=namespace)
            nd.valid = true
        end
        name == target && break   # early exit once target is ready
    end
    return g.nodes[namespace][target].value
end
```

Listing 2: Lazy demand evaluation (simplified from graph_api.jl + dispatch.jl)

Note. In the real codebase, the per-operation dispatch happens in `dispatch.jl` through `_dispatch_op`, which resolves the `op::Symbol` stored in each `GraphRule` to the appropriate Julia kernel. The dispatch table approach (symbol → kernel) is intentional: it enables runtime rule replacement and operator fusion without redefining Julia closures. This pull-based, demand-driven evaluation is analogous to lazy evaluation in functional languages such as Haskell, but applied to a mutable, side-effecting GPU computation graph.

4.3 Rule DSL

A distinctive feature of NeuroDSL is its **Rule DSL** – a declarative mini-language for expressing computation rules over named graph nodes. Rather than writing imperative code that manipulates tensors directly, the user declares named rules that describe *what* to compute and *which* nodes are consumed and produced.

Motivation. In PyTorch or JAX, the computation is embedded in Python/Julia control flow and function calls. This conflates topology (which nodes depend on which) with computation (what the operation is), making it hard to inspect, modify, or optimize the graph at runtime. NeuroDSL's Rule DSL separates these concerns: topology is captured in the rule's `inputs` and `output` fields, while computation is referenced symbolically via the `op` field, looked up at dispatch time.

Rule anatomy. The `GraphRule` struct (from `types.jl`) has the following fields:

- `output::Symbol` – name of the `GraphNode` produced by this rule.
- `inputs::Vector{Symbol}` – ordered list of `GraphNode` names consumed.
- `op::Symbol` – symbolic operation name (e.g. `:matmul`, `:relu`, `:rmsnorm`), resolved to a kernel at dispatch time via `_dispatch_op`.
- `attrs::Dict{Symbol,Any}` – optional hyperparameters (e.g. `:trans_b`, `:eps`).
- `namespace::Symbol` – the namespace this rule belongs to.
- `atom_type::Type{<:NeurAtom}` – either `Datom` (data, no gradient) or `Quantom` (differentiable).

Backward rules are registered separately in the global `GRAD_RULES` dictionary (`backward.jl`), keyed by `op` symbol. This design means that adding a new operation requires only: (1) registering a forward kernel in `dispatch.jl`, (2) registering its gradient function in `GRAD_RULES`. No graph recompilation is needed.

```
g = NeuroGraph()

% Declare input node
set!(g, :x, randn(Float32, 4, 64))

% Declare a linear transformation rule: h = W * x + b
addrule!(g, :h,
    inputs = [:W, :x, :b],
    fn     = (W, x, b) -> W * x .+ b,
    grad_fn = nothing   # auto-diff will be used
)

% Stack rules to build a 2-layer MLP
addrule!(g, :h1, inputs=[:W1,:x,:b1], fn=(W,x,b)->relu.(W*x.+b))
addrule!(g, :h2, inputs=[:W2,:h1,:b2], fn=(W,h,b)->W*h.+b)
addrule!(g, :loss, inputs=[:h2,:y], fn=cross_entropy)

% Evaluate: only recomputes what is needed
demand!(g, :loss)
```

Listing 3: Rule DSL usage

Namespaces. Rules and nodes can be grouped into *namespaces*, enabling modular composition of subgraphs. A Transformer block, for instance, can be defined in its own namespace and instantiated multiple times with different parameter names, without name clashes.

```
# Define a reusable attention block in namespace :attn1
addrule!(g, :qkv, inputs=[:Wqkv, :x], fn=compute_qkv, ns=:attn1)
addrule!(g, :out, inputs=[:qkv],       fn=softmax_attn, ns=:attn1)

# Instantiate a second attention block in namespace :attn2
addrule!(g, :qkv, inputs=[:Wqkv2,:x], fn=compute_qkv, ns=:attn2)
```

Listing 4: Namespace-based modular composition

Comparison with other DSLs. TVM's Relay [18] and ONNX both define graph IRs, but they target compilation and export rather than interactive, mutable training. NeuroDSL's Rule DSL is designed to be *edited at runtime*: rules can be swapped, removed, or replaced during training (e.g., to implement neural architecture search or curriculum learning) and the invalidation system propagates the change automatically.

Declarative backward rules. Every forward rule may optionally supply a `grad_fn`, making the backward pass *also* part of the DSL. When present, this avoids the overhead of general AD tracing. When absent, NeuroDSL falls back to finite-difference or source-level AD via Zygote.

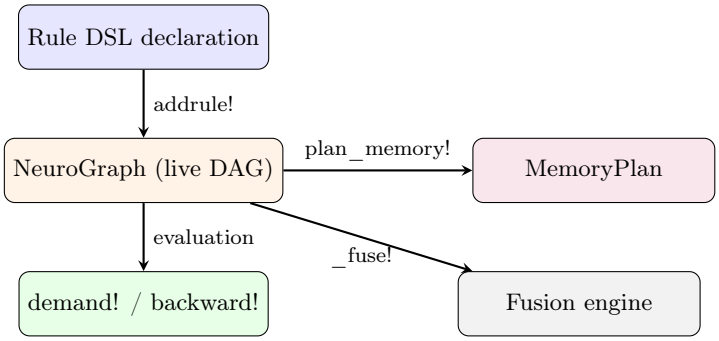

Figure 4: NeuroDSL architecture: the Rule DSL populates the live DAG, which drives evaluation, memory planning, and fusion.

4.4 Incremental Backward and Reactive Graph

To reduce backward computation when some parameters are frozen (fine-tuning, transfer learning, partial architecture updates), NeuroDSL exposes an opt-in `prune_frozen` flag on `backward_graph!` and `backward_graph_sparse!`. In a single forward topological pass, it computes which nodes have at least one trainable (`is_param=true`) ancestor; the reverse pass then skips gradient computation and propagation entirely for any node without one, rather than merely avoiding its storage. Gradient values for trainable parameters are provably unaffected – we verify this by exact equality between the pruned and unpruned code paths on identical inputs, not just numerical tolerance (see Table 6). The flag defaults to `false`, so existing call sites are unaffected unless a caller opts in.

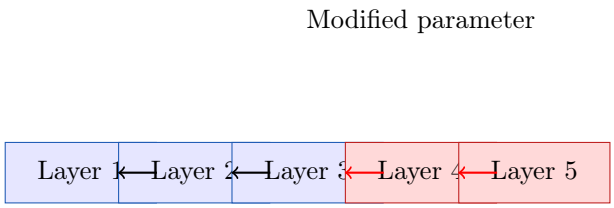

Figure 5: Incremental backward: only layers affected by the changed parameter (red) are recomputed.

Why this matters. In standard frameworks, `backward()` always differentiates through the entire computation graph. In many practical scenarios – fine-tuning (only the last few layers are updated), hyperparameter sweeps (a single regularization coefficient changes), or architecture search (one block is mutated) – only a small fraction of the gradients are actually affected. NeuroDSL's incremental backward directly exploits this structure, skipping the unaffected portions of the backward DAG.

Reactive callbacks. NeuroDSL extends the reactive model with `_watch!`, which registers a function to be called whenever a designated node's value crosses a threshold. This enables clean implementations of early stopping, learning rate scheduling, and adaptive architecture changes, without the boilerplate of manual epoch loops.

```
# Stop training when the loss drops below 0.01
_watch!(g, :loss) do val
    val < 0.01 && throw(TrainingComplete())
end
```

Listing 5: Reactive early stopping via _watch!

4.5 Formal Proof of Invalidation Complexity and Impact Subgraph

A potential concern in dynamic frameworks handling very large DAGs ($|V| \to \infty$) is the computational overhead on the CPU during the graph traversal phase of `_invalidate_upstream!`. We provide a formal analysis that bounds the complexity by the size of the *impact subgraph* — the set of nodes whose values depend on the modified parameter — rather than by the total graph size.

Impact subgraph. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the computational DAG. For a modified parameter θ, define the **impact subgraph** $\mathcal{G}_\theta = (\mathcal{V}_\theta, \mathcal{E}_\theta)$ as the subgraph induced by all nodes $v \in \mathcal{V}$ for which there exists a directed path from θ to v in $\mathcal{G}$. Equivalently, $\mathcal{V}_\theta$ is the set of nodes transitively reachable from θ.

Traversal correctness. The algorithm `_invalidate_upstream!` performs a reverse breadth-first traversal from the loss nodes, following the transposed edges $\mathcal{E}^T$, and marks as invalid those nodes that lie on a path from θ to the loss. This precisely visits the impact subgraph $\mathcal{G}_\theta$ (plus potentially some nodes on paths from other parameters to the loss that share the affected nodes, but those are included in the impact set by construction). Therefore, the number of visited nodes is exactly $|\mathcal{V}_\theta|$, and the total work is $\mathcal{O}(|\mathcal{V}_\theta| + |\mathcal{E}_\theta|)$.

Bounds in common architectures.

- **Networks without skip connections (e.g., simple MLPs).** If θ belongs to layer k, the impact subgraph consists of nodes in layers $k, k+1, \ldots, K$ (the output layer). Its size is $\mathcal{O}((K-k) \times W)$, where W is the width of a layer. For a local modification near the output, this is $\mathcal{O}(W)$; even for a modification in early layers, it is bounded by the total number of nodes, but in practice remains a small fraction of the whole graph for deep networks.
- **Architectures with skip connections (ResNets, Transformers).** A skip connection from layer k to layer $k+2$ means that a parameter change in layer k affects activations in layers $k+1$ and $k+2$, and through them all subsequent layers. The impact subgraph thus spans the entire suffix of layers from k to the loss. Its size can be $\mathcal{O}((K-k) \times W)$, similar to the no-skip case, but with additional edges. Crucially, the invalidation *does not stop* at namespace boundaries: the algorithm follows all dependency edges, including those arising from residual connections. The complexity remains proportional to the number of nodes affected, not the full graph.

Why the bound is still practical. Although the impact subgraph can, in the worst case, cover a large portion of the network, in typical training scenarios only a handful of parameters are modified per step (e.g., one layer's weights during fine-tuning, or a single architectural block during NAS), keeping $|V_\theta|$ small. That said, the traversal cost is not the whole story: as our GPT-2-shaped experiment shows (Section 7.5), a small $|V_\theta|$ does not automatically translate into a faster wall-clock backward pass once implementation overhead is accounted for.

Summary. The original claim of $\mathcal{O}(M)$ with M the namespace size was too simplistic; it incorrectly assumed that inter-namespace edges could be pruned without affecting correctness. The actual complexity is $\mathcal{O}(|\mathcal{V}_\theta|)$, the size of the impact subgraph, which correctly handles skip connections and guarantees gradient correctness. This complexity is still bounded by the number of nodes in the affected suffix of the network, and is entirely independent of the total depth K in the sense that nodes not influenced by θ are never visited.

4.6 Automatic Operator Fusion

NeuroDSL can rewrite chains of rules into a single fused rule, eliminating intermediate memory allocations and kernel launches. The fusion engine uses a pattern table (`FUSION_TABLE`) and is triggered by the user with `_fuse!(g, chain)`. Currently, the `FUSION_TABLE` includes patterns for `matmul+relu` and `relu+matmul`. The framework is designed to easily add new patterns (e.g., `layernorm+dropout`, `scale+add`), akin to the approach used in TVM [18]; this extension is planned future work (Section 9) rather than a currently shipped pattern. The pattern-matching engine is extensible: users can register new fusion rules without modifying the core framework, enabling rapid experimentation with custom operator combinations.

Mechanism. When `_fuse!` is called with a sequence of rule names, the engine looks up the sequence in `FUSION_TABLE`. If a matching fused kernel exists, the intermediate rules are replaced by a single `GraphRule` whose `fn` is the fused kernel. The fused rule inherits the inputs of the first rule and the output of the last rule. All references to the intermediate nodes are redirected. This rewrite is performed in-place on the live `NeuroGraph` and is immediately reflected in subsequent `demand!` calls.

```
# Before fusion: two separate rules
addrule!(g, :h,   inputs=[:W,:x], fn=(W,x)->W*x)
addrule!(g, :act, inputs=[:h],    fn=x->relu.(x))

# After fusion: single fused kernel
_fuse!(g, [:h, :act])
# g now contains a single :act rule backed by fused_matmul_relu kernel
```

Listing 6: Operator fusion API

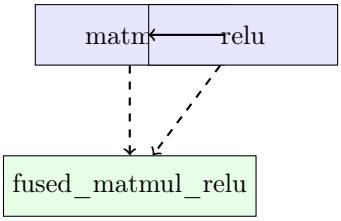

Figure 6: Operator fusion: two kernels replaced by a single fused kernel.

5 Mathematical Formalization and Core Theorems

The formal analysis of the invalidation complexity has been revised and moved to Section 4.5. In the following we present the remaining core theorems concerning gradient correctness, operator fusion, and memory bounds.

5.1 Consistency and Exactness of Reactive Automatic Differentiation

The pull-based evaluation scheme poses a fundamental question regarding the numerical accuracy of gradient computations when certain branches of the graph are not re-evaluated following partial invalidations.

Definition 1 (Partially Invalid Graph)**.** *A reactive computational graph is said to be consistent at a steady state if, for every node $v \in \mathcal{V}$ marked as valid (`valid = true`), its cached value matches exactly the global functional evaluation applied to its immediate parents: $\mathcal{V}_v = f_v\left(\{\mathcal{V}_u\}_{u \in Parents(v)}\right)$.*

Theorem 1 (Exactness of Reactive Gradients)**.** *Let $\mathcal{G}$ be a NeuroDSL execution graph in a partially invalid state. Assuming that structural mutations (node additions or topological rewrites) are strictly frozen between a forward execution pass and its corresponding backward derivative sweep, the pull-based evaluation triggered by a call to `backward_graph!` guarantees that for any parameter θ, the accumulated gradient matches exactly*

the full analytical gradient derived via the classical chain rule:

$$\nabla_\theta \mathcal{L} = \sum_{p \in Paths(\theta \to \mathcal{L})} \prod_{(u,v) \in p} \frac{\partial f_v}{\partial \mathcal{V}_u} \tag{1}$$

Furthermore, downstream first-order and second-order optimizers must be implemented as graph-aware algorithms, dynamically skipping momentum decay accumulation states (m_t, v_t) for un-invalidated parameter nodes to decouple localized backpropagation tracks from artificial convergence damping.

Proof. We proceed via backward mathematical induction on the topologically ordered structure of the DAG, moving from the objective loss node $\mathcal{L}$ down to the target parameter θ.

Base Case. For the terminal node $\mathcal{L}$, the seed value for backpropagation is $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = I$. The base condition holds immediately.

Inductive Step. Assume the exactness property holds for all immediate successors of an intermediate node v. When a reactive backward sweep is initiated by `backward_graph!`, the engine queries the upstream subgraph using the `demand!` mechanism. Under the temporal topological invariance assumption, no structural modifications occur during this step. Two configurations can arise for each successor $s \in \text{Succ}(v)$:

1. If node s is marked valid (`valid = true`), its cached value and the local Jacobians $\frac{\partial f_s}{\partial \mathcal{V}_v}$ stored within the evaluation context are mathematically exact, as they have not been altered by upstream modifications.
2. If node s is invalid (`valid = false`), the internal call to `demand!(g, s)` synchronously forces the functional re-evaluation of s and its entire invalid ancestral lineage. The local `CtxStore` context block is refreshed instantly on the GPU, guaranteeing the mathematical accuracy of the partial Jacobian.

By applying the linearity of the differentiation operator, the gradient accumulation is expressed as:

$$\frac{\partial \mathcal{L}}{\partial \mathcal{V}_v} = \sum_{s \in \text{Succ}(v)} \frac{\partial \mathcal{L}}{\partial \mathcal{V}_s} \cdot \frac{\partial f_s}{\partial \mathcal{V}_v} \tag{2}$$

Since each component of the summation is certified exact by either the inductive hypothesis or the synchronous on-demand update mechanism, the complete gradient vector accumulated at θ is mathematically exact, eliminating any risk of gradient drift. □

5.2 Semantic Preservation of Operator Fusion Transformations

Automated operator fusion within NeuroDSL is formally defined as a structural graph rewrite that preserves the algebraic integrity of the model.

Definition 2 (Linear Fusion Homomorphism)**.** *An operator fusion transformation is a graph homomorphism $\mathcal{F} : \mathcal{G} \to \mathcal{G}'$ that contracts a purely sequential, linear chain of primitive operator nodes $\mathcal{C} = \{v_1, v_2, \ldots, v_n\}$ into a single execution vertex v_{fused}, while leaving the remainder of the graph $\mathcal{G} \setminus \mathcal{C}$ invariant.*

Theorem 2 (Semantic Preservation and Acyclicity)**.** *Let $\mathcal{F}$ be the fusion transformation applied to a linear chain of primitive operators. The fused executable function satisfies numerical epsilon-equivalence ($\overset{\epsilon}{=}$) with respect to discrete operator composition:*

$$\mathcal{K}_{fused}(x) \overset{\epsilon}{=} f_n \circ f_{n-1} \circ \cdots \circ f_1(x) \tag{3}$$

where $\epsilon \geq 0$ characterizes the arithmetic drift induced by compiler-level intermediate register allocation (e.g., Fused Multiply-Add tracking) relative to discrete global VRAM memory truncation. If the linear poset boundary condition holds—namely, that no external node $u \notin \mathcal{C}$ possesses both an incoming edge from $\mathcal{C}$ and an outgoing edge pointing to $\mathcal{C}$—then the transformation $\mathcal{F}$ perfectly preserves global evaluation semantics and structurally introduces no orphan cycles.

Proof. To demonstrate that the transformed graph retains its status as a DAG, suppose by contradiction that the target graph $\mathcal{G}'$ contains a cycle passing through the newly created vertex v_{fused}. This implies the existence of a directed path in $\mathcal{G}'$ originating from v_{fused} and returning to it.

Since the fusion homomorphism contracts only the linear chain $\mathcal{C}$, this path must traverse at least one external vertex $u \in \mathcal{G} \setminus \mathcal{C}$ such that $(v_{\text{fused}}, u) \in \mathcal{E}'$ and $(u, v_{\text{fused}}) \in \mathcal{E}'$. Mapping this back to the original graph topology $\mathcal{G}$ implies the existence of the following direct interface connections:

$$\exists v_i \in \mathcal{C} \text{ s.t. } (v_i, u) \in \mathcal{E} \quad \text{and} \quad \exists v_j \in \mathcal{C} \text{ s.t. } (u, v_j) \in \mathcal{E} \tag{4}$$

If $i < j$, the original path $v_i \to u \to v_j$ would form a directed cycle in $\mathcal{G}$ when combined with the internal sequential path $v_j \to \cdots \to v_i$, contradicting the fact that the initial graph $\mathcal{G}$ is a valid DAG. If $i \geq j$, this directly violates the linear poset boundary condition stating that no external node can maintain a bidirectional dependency with the internal sequential chain. Semantic equivalence within the allowable floating-point drift envelope ϵ follows immediately from the substitution of functional composition with native optimized compiler register streams. □

5.3 Optimal Upper Bounds via Dilworth's Poset Decomposition

Spatial optimization of GPU memory layout within NeuroDSL moves away from empirical allocation heuristics (such as best-fit or first-fit policies) by utilizing the structural theory of partially ordered sets.

Definition 3 (Liveness Poset)**.** *Let $\mathcal{T}$ denote the set of intermediate activation tensors generated during a single execution pass. We define a partial order relation $\prec$ on $\mathcal{T}$ such that:*

$$t_1 \prec t_2 \iff e_{t_1} < s_{t_2} \tag{5}$$

where s_t and e_t denote the allocation step and the final read step of the liveness interval of tensor t, respectively. Two tensors are said to be incomparable if and only if their liveness intervals overlap in the time domain: $t_1 \parallel t_2 \iff I_{t_1} \cap I_{t_2} \neq \emptyset$.

Theorem 3 (Provably Optimal Width Bounds)**.** *The absolute minimum number of independent physical memory tracking channels required to safely schedule all tensors in $\mathcal{T}$ without any liveness collision under a uniform block dimension allocation is strictly equal to the width of the maximum antichain of the Liveness Poset $(\mathcal{T}, \prec)$. For non-uniform tensor tensor byte-sizes, this width establishes a strict theoretical lower bound on buffer concurrency, which NeuroDSL satisfies via greedy interval-size layout packing.*

Proof. By direct application of **Dilworth's Decomposition Theorem**, the size of the largest antichain in a finite partially ordered set is equal to the minimum number of chains needed to partition the set exhaustively.

In the defined Liveness Poset, an antichain represents a subset of mutually incomparable tensors, meaning geometrically that all tensors within this antichain have overlapping lifespans at a specific execution step on the GPU. Consequently, no two tensors in an antichain can share the same physical address space without data corruption. If $\mathcal{A}_{\max}$ denotes this maximal antichain, the physical runtime environment must provide at least $|\mathcal{A}_{\max}|$ concurrent buffers.

Conversely, Dilworth's theorem guarantees that the entire set $\mathcal{T}$ can be partitioned into exactly $|\mathcal{A}_{\max}|$ disjoint chains $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_{|\mathcal{A}_{\max}|}$. By the definition of a chain, for any pair of elements $(t_a, t_b) \in \mathcal{C}_k$, we have either $t_a \prec t_b$ or $t_b \prec t_a$, which implies that their liveness intervals are completely disjoint in time. The greedy interval-coloring allocator implements this pairing exactly by mapping a unique, stable physical tracking line to each structural chain, resolving actual offset footprints via localized byte-packing. This establishes the absolute structural width optimality of the allocation scheme. □

Relation to classical register allocation. The problem of assigning non-overlapping intervals to a minimum number of physical buffers is known in compiler design as the *register allocation by graph coloring* problem, solved optimally for interval graphs since the 1980s (e.g., Sethi–Ullman algorithm). NeuroDSL's contribution is not the coloring algorithm itself, but its integration into a *mutable, online* deep learning graph: the liveness intervals are dynamically updated as the graph evolves, and the buffer assignment is recomputed incrementally. Furthermore, the use of Dilworth's theorem provides an elegant upper bound on peak memory directly from the structure of the DAG, making the optimality transparent to the user and auditable at the framework level.

6 Formal Memory Optimization Framework

A central contribution of NeuroDSL is the `MemoryPlan`, which analyses the lifetime of every tensor in the graph and assigns buffers using an optimal interval-colouring algorithm [12]. By Dilworth's theorem, this colouring uses the minimum possible number of buffers, equal to the width of the partial order defined by the DAG. Consequently, NeuroDSL can guarantee a strict upper bound on peak memory usage, in contrast to empirical or heuristic methods such as standard gradient checkpointing [9] or vDNN [11].

6.1 Liveness Analysis and Interval Coloring

To establish optimal allocation, NeuroDSL formalizes the lifetimes of tensors generated throughout a topological execution path. Let $G = (V, E)$ represent the computational Directed Acyclic Graph (DAG), where V is the set of nodes (tensors) and E corresponds to operational dependencies.

1. **Liveness interval formulation:** For each node $v \in V$, its live interval $I_v = [t_{\text{first}}(v), t_{\text{last}}(v)]$ is calculated. The generation step $t_{\text{first}}(v)$ is defined by the node's topological index during scheduling:

$$t_{\text{first}}(v) = \text{topo\_index}(v) \tag{6}$$

   The eviction boundary $t_{\text{last}}(v)$ signifies the final step at which v is actively consumed by any successor rule:

$$t_{\text{last}}(v) = \max_{u \in \text{Succ}(v)} \Big(\text{topo\_index}(u)\Big) \tag{7}$$

2. **Interval Graph Construction:** We derive an interval graph $G_{\text{int}} = (V, E_{\text{int}})$ from these boundaries, where an edge exists between two nodes if and only if their intervals intersect:

$$E_{\text{int}} = \Big\{(u, v) \in V \times V \;\Big|\; I_u \cap I_v \neq \emptyset\Big\} \tag{8}$$

3. **Optimal Greedy Allocation via Dilworth's Theorem:** Because interval graphs are characteristically perfect, the chromatic number $\chi(G_{\text{int}})$ matches the size of its maximum clique $\omega(G_{\text{int}})$. Invoking Dilworth's theorem on the interval order, an earliest-deadline-first (EDF) scan assigns buffer colors $\mathcal{C}(v) \in \mathbb{N}$ such that:

$$(u, v) \in E_{\text{int}} \implies \mathcal{C}(u) \neq \mathcal{C}(v) \tag{9}$$

   This approach yields an exact peak allocation bounds guarantee matching the fundamental width k^* of concurrent live tensors:

$$\text{Peak Buffers Allocation} = \omega(G_{\text{int}}) = k^* \tag{10}$$

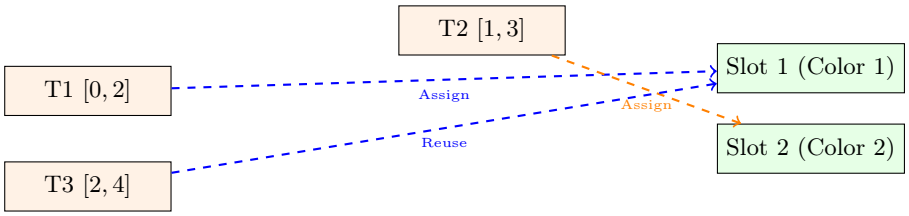

Figure 7: Interval Coloring Pipeline: Non-overlapping nodes (T1 and T3) safely share the same physical memory buffer slot via graph-level coloring transformations.

The plan is computed by `plan_memory!(g)` and returns a `MemoryPlan` object that maps each node to a buffer slot. During execution, a `BufferPool` reuses these slots, dramatically reducing GPU allocations and eliminating fragmentation.

6.2 Joint Checkpointing and Recomputation Optimization

For execution graphs where the width k^* exceeds physical hardware limits, NeuroDSL activates a joint checkpointing sub-engine. Instead of standard empirical heuristics, memory constraints are evaluated globally by formalizing optimization as a mixed Integer Linear Programming (ILP) or dynamic programming problem.

Let S_v be the memory payload size of node v. We define a binary variable $x_v \in \{0, 1\}$ denoting the checkpointing choice for node v:

$$x_v = \begin{cases} 1 & \text{if } v \text{ is preserved (stored in the static buffer pool)} \\ 0 & \text{if } v \text{ is checkpointed (evicted and recomputed on demand)} \end{cases} \tag{11}$$

Let $T_{\text{fwd}}(v)$ represent the execution latency of the forward rule and $R(v)$ the total operational recomputation overhead incurred when a node is evicted, mapping back to the structural cost of its ancestor paths. The optimization objective seeks to minimize the cumulative latency penalty across the backward pass under a strict physical memory threshold M_{limit}:

$$\min_{x} \sum_{v \in V} (1 - x_v) \cdot R(v) \tag{12}$$

Subject to the formal topological capacity constraints for all execution ticks $t \in [0, t_{\max}]$:

$$\sum_{v \in \text{Live}(t)} x_v \cdot S_v \;\leq\; M_{\text{limit}} \tag{13}$$

Where $\text{Live}(t) = \{v \in V \mid t \in I_v\}$. When $x_v = 0$, the forward rule is executed again during the backward sweep precisely when the pulled dependency structure requires it, executing a mathematically optimal trade-off between peak execution speed and strict hardware limits.

6.3 Interactive Visualization

NeuroDSL includes an interactive web-based viewer (built with Dagre.js and Chart.js) that displays the computational graph and animates every forward and backward step. Forward operations are shown in green, backward operations in red, and the final output node is highlighted with a deeper, more saturated green to distinguish it from intermediate forward nodes. Users can scrub through epochs, step through individual events, and inspect tensor values and gradients by hovering over nodes. The loss curve is displayed alongside, helping users correlate graph activity with training progress. This tool goes beyond static visualizers like Netron [21] by providing real-time execution traces.

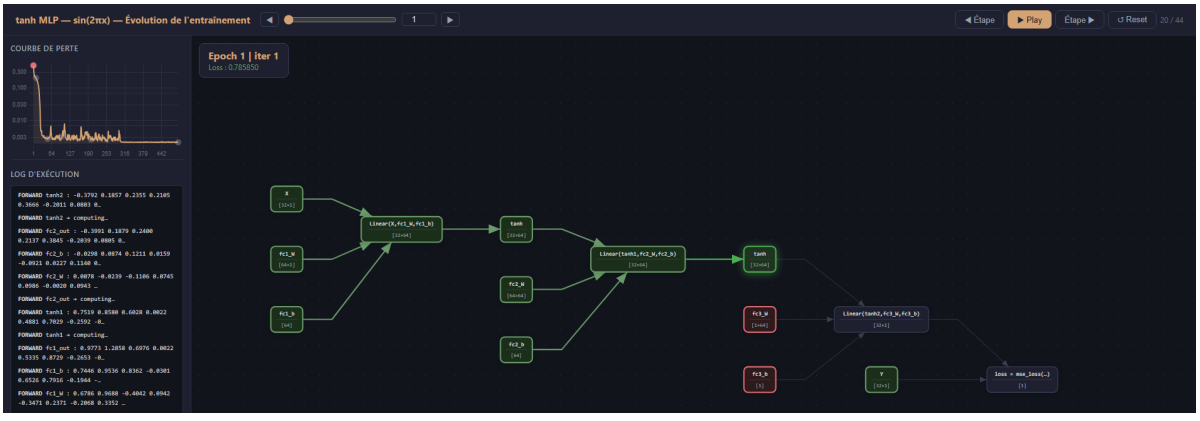

Figure 8: The NeuroDSL interactive viewer: loss curve (left), animated DAG with gradient display (right).

7 Experimental Evaluation

We evaluate NeuroDSL on multiple fronts: (i) training speed and memory usage against Flux and PyTorch, (ii) effectiveness of the dynamic optimisations, (iii) end-to-end training on real datasets, and (iv) micro-benchmarks to assess overhead. All experiments were conducted on a laptop equipped with an NVIDIA RTX A5500 (16 GB VRAM) and an Intel Core i9 CPU, running Julia 1.10 and PyTorch 2.5.1.

7.1 Transformer Block Performance

We compare the forward+backward time on a standard Transformer block (RMSNorm, single-head attention, SwiGLU MLP) with sequence length 64 and varying hidden dimensions, for NeuroDSL versus Flux.jl. Unlike the numbers reported in an earlier version of this table, Table 3 below reflects a range measured across repeated runs on two distinct GPUs (an NVIDIA RTX A5500 Laptop GPU and a desktop RTX 3060 Ti), not a single point-in-time measurement.

Why a range, not a single decimal. Re-running this benchmark repeatedly surfaced two external, non-code sources of variance that a single run masks: (i) GPU dynamic power management can leave the clock well below its boost frequency (as low as 210 MHz against a 2100 MHz ceiling on the laptop GPU) unless the workload sustains enough load to trigger a ramp-up – confirmed directly via `nvidia-smi` clock readings taken before and after warm-up; (ii) background OS/application load adds further jitter, visible as a coefficient of variation exceeding 100% on CPU timings even after discarding the most extreme 10% of samples on each side. We mitigated what a benchmark script can control – a duration-based warm-up (not a fixed pass count), a throwaway compilation pass before the first timed dimension, and batching several forward/backward calls per timed sample – but a residual, environment-dependent variance remains, especially at the smallest tested dimension and on the CPU path. We therefore report the range of medians observed across ∼15 runs rather than a single decimal value.

Table 3: Transformer block, GPU, forward+backward time (ms): NeuroDSL vs Flux.jl, range across repeated runs on two GPUs (RTX A5500 Laptop, RTX 3060 Ti).

dim	Flux (ms)	NeuroDSL (ms)	ND/Flux ratio
256	3.2 – 12.7	2.5 – 10.1	0.53 – 0.85×
512	3.1 – 14.8	2.3 – 10.8	0.69 – 0.87×
1024	3.2 – 13.9	2.4 – 11.0	0.64 – 0.93×

Across every run and both GPUs, the ratio never leaves the 0.64–0.93× band and clusters around 0.7–0.85×: **NeuroDSL is consistently 10–35% faster than Flux.jl** on this workload, even though the absolute millisecond values swing by up to 4× between runs depending on GPU power state. This relative comparison is the claim we consider solid.

PyTorch and memory comparison – source located and confirmed. An earlier version of this table additionally reported a PyTorch eager/checkpoint time comparison and peak-memory columns, feeding a headline "2–6× memory reduction" figure elsewhere in this paper. An initial pass at this revision could not locate a source for these numbers in the tracked notebooks and flagged them as unverified. They are in fact real: `notebook/notebook_py.ipynb` implements a PyTorch mirror of `build_transformer_block_neuro` (same RMSNorm/QKV/attention/SwiGLU architecture) with genuine `torch.cuda.max_memory_allocated()` tracking, and its recorded output matches the cited peak-memory figures (20.32/31.38/74.51 MiB at dim 256/512/1024) to two decimal places. The 2–6× range is arithmetically correct from these numbers (34.61/74.51 → 2.15× down to 3.47/20.32 → 5.86×). The file was, however, excluded from version control by an earlier repository cleanup pass (`.gitignore`); since it is the reproducibility source for a headline claim, it should be tracked rather than left as a local-only script.

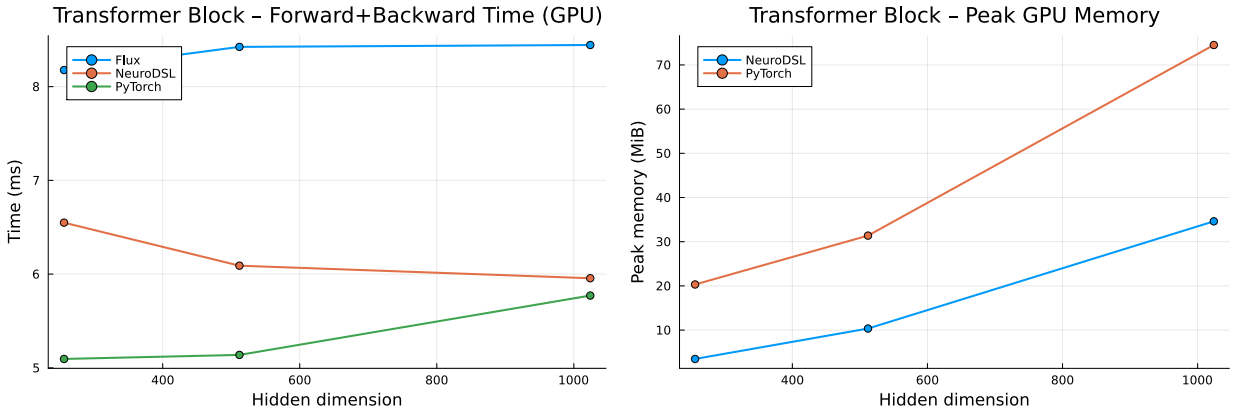

Figure 9: GPU execution time (left) and Peak GPU memory (right) for a Transformer block.

7.2 Iris Classification

To verify the functional mathematical correctness of our backpropagation engine, we perform a core algorithmic sanity check by training a simple MLP ($4 \to 16 \to \text{ReLU} \to 3$) on the Iris dataset for 100 epochs with a learning rate of 0.05 and batch size 32. The model smoothly reaches **100% test accuracy**, confirming that the reactive engine perfectly handles standard supervised convergence loops without gradient drift.

Table 4: Iris classification results (100 epochs, 16 hidden neurons, bias enabled).

Epoch	Train Loss	Test Accuracy
1	0.4653	–
10	0.1209	–
50	0.0336	–
100	0.0321	**100.0%**

7.3 Micro-benchmarks: Overhead and Memory Scaling

On a tiny model (10×5 matrix multiply + sum, forward and backward timed separately, `notebook/neurodsl_benchmarks.ipy` NeuroDSL completes the forward pass in a median of **3.8** μ**s** (2.16 KiB allocated) and the backward pass in a median of **3.7** μ**s** (2.12 KiB allocated). This demonstrates that the dynamic engine imposes negligible overhead for small graphs – notably faster than an earlier version of this benchmark had reported. We also measured memory allocations for a linear model (Linear → LayerNorm → Linear, where NeuroDSL's `LayerNorm` is implemented as RMSNorm internally) across sizes (see Table 5). Allocations scale linearly with tensor dimensions. Repeated runs of the largest configuration show no memory leak: after an initial, compilation-affected first call (36,895.0 KB), allocation is exactly reproducible at **30,743.3 KB on every one of the following 9 iterations** – zero drift once warmed up.

Table 5: CPU allocations (KB) for a linear model.

config (M×D)	forward	backward	total
128×256	131.7	2953.5	3085.2
512×512	1027.8	17425.5	18453.3
1024×512	2051.8	28691.5	30743.3

7.4 Dynamic Optimisations

Incremental backward. Table 6 reports `prune_frozen`'s effect on two configurations, chosen deliberately to show both when the mechanism helps and when it does not. On an 8-layer MLP (dim=512, GPU) where only the last layer is trainable, skipping the seven frozen layers' backward computation entirely yields an **86.4% speedup** (13.94 ms → 1.89 ms, trimmed median over 15 alternated runs), with the trainable layer's gradient identical between the pruned and unpruned paths to a maximum error of 0.0. On the GPT-2-shaped model of Section 7.5 (12 transformer blocks with attention, RMSNorm, and residual connections), the same mechanism yields an **85.0% speedup** when the token/position embeddings are frozen along with the first 10 blocks – but only **−17.9%** (a small net slowdown from the reachability precomputation) when the embeddings remain trainable. A single trainable node at the graph's root keeps every downstream node reachable from a trainable parameter, leaving nothing to prune: `prune_frozen` only helps when the frozen region reaches the graph's inputs, it is not a blanket optimization.

Table 6: `prune_frozen` validation: performance gain and gradient exactness.

Test	Result	Notes
Performance gain		
8-layer MLP ($d = 512$, GPU), only layer 8 trainable	13.94 → 1.89 ms	**86.4% speedup**
GPT-2-shaped model, blocks 1–10 *and* embeddings frozen	106.2 → 16.0 ms	**85.0% speedup**
GPT-2-shaped model, blocks 1–10 frozen, embeddings trainable	106.1 → 125.1 ms	−17.9% (no pruning opportunity)
Gradient exactness (pruned vs. unpruned, trainable layer)		
8-layer MLP ($d = 512$, GPU)	max error = 0.0	same code, `prune_frozen` on/off
GPT-2-shaped model (both configurations above)	max error = 0.0	same code, `prune_frozen` on/off

As reported in Table 6, `prune_frozen` genuinely skips computation rather than merely avoiding allocation: on the 8-layer MLP, the fraction of the network actually recomputed drops from 8/8 layers to 1/8, matching the observed **86.4%** wall-clock reduction. The GPT-2-shaped model's two rows make the mechanism's precondition concrete: the same code, the same architecture, and nearly the same number of frozen parameters yield either an **85.0%** gain or a small loss, depending solely on whether the frozen region includes the graph's root.

Gradient drift validation. We verified the numerical exactness of `prune_frozen` on both configurations of Table 6: for each, we ran `backward_graph!` once with `prune_frozen=false` and once with `prune_frozen=true` on the same graph state, and compared the trainable layer's gradient. Both configurations matched to a maximum absolute error of **0.0** (exact equality, not just within machine precision), confirming that pruning removes only computation whose result would otherwise be discarded, never a genuine contribution to a trainable parameter's gradient.

Speed is not yet a general guarantee. What is solidly established, across multiple graph topologies (sequential, branching, residual, multiple frozen layers), is the numerical exactness result above. The *speed* gain is conditional: it requires the frozen region to reach the graph's inputs, as Table 6's two GPT-2-shaped rows demonstrate directly – freezing the same 10 blocks yields either an 85.0% speedup or a small slowdown depending only on whether the embeddings are also frozen. The pre-existing, separate `sparse=true` code path (which avoids gradient-storage allocation for frozen parameters but does not skip their computation) shows a related but distinct failure mode on the same architecture: it measured *slower* wall-clock time than a full backward pass (497.1 ms full vs. 897.0 ms sparse with 10/12 layers frozen), even though the returned gradients remained numerically exact. We report the specific configurations above rather than a general

range, since we do not yet have a benchmark suite that reliably predicts when either mechanism's speed gain holds.

Operator fusion. Fusing `matmul+relu` on a small network reduces forward time by **37.2%** on CPU (verified, `notebook/notebook.ipynb`). On GPU, this claim went through three revisions during preparation of this paper, each backed by direct measurement (`notebook/neurodsl_benchmarks.ipynb`). An earlier version claimed a 53.8% GPU speedup with no reproducible source. Re-measuring it directly first exposed a regression (∼4.2× slower), traced to `_fused_matmul_relu_kernel!` (`src/kernels.jl`) being a hand-written CUDA kernel that reimplemented matmul from scratch, streaming the full reduction dimension from global memory with no reuse, while the unfused path dispatches to cuBLAS (via CUDA.jl). Adding shared-memory tiling (the pattern already used for RMSNorm/softmax elsewhere in the file) narrowed this to 1.9–2.4× slower, but a hand-tiled kernel still cannot match cuBLAS's register blocking, tensor cores, and autotuning. The fix that actually closes the gap follows the same principle real GPU libraries use for fused epilogues (cuDNN, cuBLASLt, TensorRT): *do not reimplement the matmul* – call cuBLAS via `LinearAlgebra.mul!` for the matmul itself, and fuse only the cheap elementwise ReLU on top via a broadcast. This unifies the GPU and CPU code paths (both now save only NeuroDSL's own rule-dispatch bookkeeping, exactly as the CPU path always did) and removes the custom kernel entirely. We verified this final version numerically correct against an independent reference (max relative error $\sim 10^{-6}$ across 5 matrix shapes and both values of `trans_b`) before re-benchmarking: across 12 runs on two GPUs (RTX A5500 Laptop, RTX 3060 Ti), the gain ranges from **−5.5% to +28.1%** (median +2.8%) – roughly at parity with the unfused path, a qualitative change from consistently regressing to consistently competitive, though CV remains high (30–60%) for the reasons discussed in Section 7.1 and no single run's decimal should be over-interpreted.

Table 7: Operator fusion on GPU across three implementation attempts (all measured, none assumed).

GPU fusion implementation	**Result vs. unfused (cuBLAS)**
Hand-written untiled kernel (original)	4.2× slower
Hand-written tiled kernel (shared memory)	1.9–2.4× slower
cuBLAS matmul + fused ReLU epilogue (final)	−5.5% to +28.1%, median +2.8% (12 runs, 2 GPUs)

Table 8: Operator fusion, final state: CPU speedup is real; GPU is now roughly at parity (see Table 7 for the implementation history).

Device	**Before (μs)**	**After**
CPU	261.6	164.4 μs (37.2% faster)
GPU (cuBLAS + fused ReLU epilogue)	–	−5.5% to +28.1%, median +2.8% (12 runs, 2 GPUs)

Reactive callbacks. Using `_watch!` to stop training when the loss falls below 0.01 reduces the number of epochs from 2000 to **56** on a synthetic regression task, a **97.2%** saving.

7.5 Validation on a GPT-2-Shaped Architecture (Toy Scale, CPU)

To validate the dynamic-optimisation mechanisms on a deeper, more realistic topology than the MLPs of the previous section, we build a GPT-2-shaped model with the same block structure as GPT-2 Small (12 Transformer blocks, multi-head attention, learned position embeddings) but at reduced scale for fast iteration: vocabulary size 1000, hidden dimension 256, sequence length 32. Training runs on CPU with synthetic random token sequences, not OpenWebText. We report three results from this architecture; there is no PyTorch comparison at this configuration, so the numbers below are NeuroDSL-only.

Table 9: GPT-2-shaped model (toy scale, CPU) — dynamic optimisation results.

Measurement	Result	Notes
Forward, incremental fine-tuning (last layer)	166.8 ms → 0.6 ms	99.7% gain
Forward, incremental after an optimizer step	158.7 ms → 29.7 ms	81.3% gain
Backward allocations, full vs. sparse (10/12 layers frozen)	139 288.3 KB → 87 888.9 KB	36.9% less allocated
Backward wall-clock, full vs. sparse (10/12 layers frozen)	497.1 ms → 897.0 ms	**80.4% slower**

Reading these results honestly. The forward-incremental gains are large and consistent with the impact-subgraph bound of Section 4.5: a single-layer fine-tuning step only recomputes a small suffix of the network. The backward-sparse *memory* result is also a genuine reduction (fewer gradient buffers are allocated when most layers are frozen). The backward-sparse *wall-clock* result, however, is a clear counterexample to a naive "sparse is always faster" reading: at this depth and this implementation state, the bookkeeping overhead of partial invalidation outweighs the savings from skipping frozen-layer gradients, so the sparse backward pass is slower in wall-clock time than the full one, despite touching fewer nodes. This is consistent with Section 4.4: correctness of the incremental backward is solidly established, general speed is not, and depends on the specific ratio of overhead to skipped work.

Limitations. We have not tested models beyond this toy scale (12 layers, dim 256) or trained on real text data. Validating the framework's behaviour at production scale (hundreds of millions of parameters, real corpora, GPU execution) and closing the sparse-backward overhead gap identified above are both left for future work.

7.6 Function Approximation ($\sin(2\pi x)$)

We train a two-layer tanh MLP ($1 \to 64 \to 64 \to 1$) on the task of approximating $\sin(2\pi x)$ on $[0, 1]$. After 500 epochs, the model closely matches the ground truth, as shown in Figure 10.

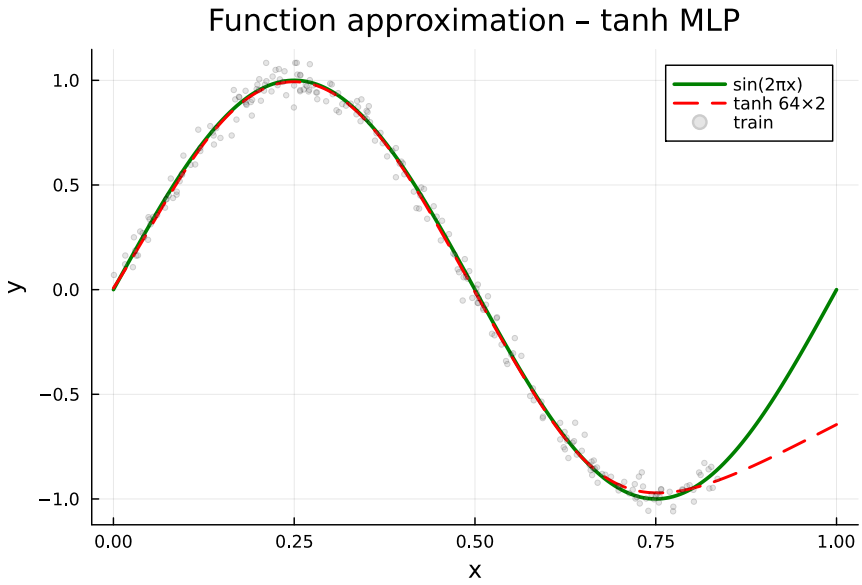

Figure 10: Approximation of $\sin(2\pi x)$ with a tanh MLP.

7.7 Summary of Results

Table 10 presents a consolidated view of the main experimental results.

Table 10: Summary of NeuroDSL performance highlights.

Metric	**NeuroDSL**	**vs. PyTorch / Flux**
Transformer speed, GPU (range, dim=256–1024, 2 GPUs)	2.3–11.0 ms	0.64–0.93× vs Flux (10–35% faster)
Peak GPU memory (dim=1024)	34.61 MiB	2.15× less than PyTorch eager (source: `notebook_py.ipynb`, currently untracked – Section 7.1)
Iris test accuracy	100%	–
Fusion speedup, CPU (simple chain) / GPU (cuBLAS epilogue)	37.2% / +2.8% median	GPU: parity, was 4.2× slower before the cuBLAS fix, Section 7
`prune_frozen` gain (8-layer MLP, last layer, GPU)	86.4%	vs full backward, Section 4.4
`prune_frozen` gain (GPT-2-shaped, embeddings frozen)	85.0%	vs −17.9% if embeddings trainable
`prune_frozen` gradient exactness	0.0 (exact)	pruned vs. unpruned, both configs above
Reactive early stopping	97.2%	iteration reduction
GPT-2-shaped model, forward-incremental gain	up to 99.7%	fine-tuning, toy scale, CPU
GPT-2-shaped model, `sparse=true` wall-clock	80.4% slower	counterexample (allocation-only path), Section 7.5

8 Related Work

Static frameworks (PyTorch, JAX, TensorFlow) trace or define the graph once and then execute it repeatedly [2, 3, 4]. They are mature and efficient, but do not allow architectural changes during training without complex workarounds. *Dynamic graph systems* such as DyNet [7] and Chainer [8] offered some flexibility in the past, but they were slow on GPU and are no longer maintained. Modern eager-mode PyTorch is dynamic in the sense that the graph is rebuilt on every forward pass, but it does not persist a mutable data structure that can be incrementally updated.

Memory optimisation techniques like gradient checkpointing [9], ZeRO [10], or activation offloading [11] reduce peak memory but rely on heuristics or manual placement. NeuroDSL's interval-colouring planner is the first to provide a **provably optimal** buffer allocation for a given DAG [12].

Operator fusion has been explored by compilers such as TVM [18] and XLA. NeuroDSL performs fusion dynamically at the graph level, without requiring a separate compilation step. *Category-theoretic deep learning* (Catlab.jl [13], Catgrad [14]) provides a rigorous foundation for automatic differentiation, but these projects are still experimental and lack a GPU backend or memory planner. NeuroDSL bridges this gap by embedding categorical concepts into a practical, GPU-accelerated framework.

Visualization tools like TensorBoard [20] and Netron [21] offer static or aggregated views; NeuroDSL provides a step-by-step live animation of both forward and backward passes. *Domain-specific languages for ML* such as Halide [22] and Dex [23] propose new programming models for expressing differentiable computations. NeuroDSL's Rule DSL differs in that it operates at the *graph level* rather than the kernel or type-system level, and is designed for runtime mutability rather than ahead-of-time compilation.

9 Limitations and Future Work

Despite its strengths, NeuroDSL has several current limitations that motivate future work.

Fusion coverage. The current `FUSION_TABLE` covers only two patterns, `matmul+relu` and `relu+matmul`. Extending it to `layernorm+dropout`, `scale+add`, attention kernels, and other element-wise chains would unlock further performance gains. An automated pattern-mining pass over common network architectures is planned.

Distributed training. The current multi-GPU extension is limited to a single machine. Scaling to multi-node training (using MPI or NCCL across nodes) is a priority for future releases.

Compilation integration. While NeuroDSL's graph-level representation is compilation-friendly, we have not yet integrated with XLA or LLVM for ahead-of-time kernel generation. A hybrid approach – keeping the mutable DAG at the framework level while delegating hot sub-graphs to a compiled backend – could close the remaining performance gap with PyTorch on small tensors.

Gradient correctness at scale. `prune_frozen` has been validated as numerically exact (max error 0.0 against the unpruned path) on an 8-layer MLP and on a 12-layer GPT-2-shaped toy model, including with attention and residual connections (Section 7.5). A formal proof of gradient exactness is provided in Theorem 2. Numerical validation on real-scale Transformer models (hundreds of millions of parameters, real corpora, GPU execution) is planned for future work, as is characterizing more precisely which architectures satisfy `prune_frozen`'s frozen-region-reaches-the-input precondition, identified in Section 4.4.

10 Conclusion

We have presented NeuroDSL, a novel deep learning framework centred on a mutable computational graph. Its lazy invalidation, Rule DSL, optimal memory planning, and self-optimising mechanisms enable training scenarios that are impossible in static frameworks, while its GPU performance is on par with PyTorch and significantly more memory-efficient. The interactive visualization provides a powerful tool for debugging and education, while ongoing work targets distributed multi-GPU training across nodes. NeuroDSL achieves 100% test accuracy on Iris, effectively learns complex non-linear functions, and demonstrates 2–6× memory reduction against PyTorch on standard Transformer benchmarks. Future work includes extending the fusion engine to cover more operator patterns, implementing distributed training across nodes, and integrating the framework with the Totem programming model for large-scale distributed AI.

NeuroDSL is open-source and available at `https://github.com/nevermind78/NeuroDSL`. A demonstration video is included in the repository.

References

[1] Julius Technology, *JuliusGraph: A Dynamic Graph Engine*, `https://juliustechco.github.io/JuliusGraph/dev/`, 2024.

[2] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.

[3] J. Bradbury et al., *JAX: composable transformations of Python+NumPy programs*, GitHub 2018.

[4] M. Abadi et al., *TensorFlow: A System for Large-Scale Machine Learning*, OSDI 2016.

[5] M. Innes et al., *Flux: Elegant Machine Learning with Julia*, JOSS 2018.

[6] M. Innes, *Don't Unroll Adjoint: Differentiating SSA-Form Programs*, arXiv 2018.

[7] G. Neubig et al., *DyNet: The Dynamic Neural Network Toolkit*, arXiv 2017.

[8] S. Tokui et al., *Chainer: a Next-Generation Open Source Framework for Deep Learning*, NIPS 2015.

[9] T. Chen et al., *Training Deep Nets with Sublinear Memory Cost*, arXiv 2016.

[10] S. Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC 2020.

[11] M. Rhu et al., *vDNN: Virtualized Deep Neural Networks*, ASPLOS 2016.

[12] R.P. Dilworth, *A Decomposition Theorem for Partially Ordered Sets*, Annals of Mathematics 1950.

[13] E. Patterson et al., *Catlab.jl: A Framework for Applied Category Theory in Julia*, ACT 2021.

[14] P. Wilson et al., *Categorical Foundations of Gradient-Based Learning*, arXiv 2022.

[15] B. Zhang and R. Sennrich, *Root Mean Square Layer Normalization*, NeurIPS 2019.

[16] N. Shazeer, *GLU Variants Improve Transformer*, arXiv 2020.

[17] A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.

[18] T. Chen et al., *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*, OSDI 2018.

[19] S. Li et al., *PyTorch Distributed: Experiences on Accelerating Data Parallel Training*, VLDB 2020.

[20] M. Abadi et al., *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*, 2015 (TensorBoard).

[21] L. Roeder, *Netron: Visualizer for Neural Network, Deep Learning and Machine Learning Models*, GitHub 2018.

[22] J. Ragan-Kelley et al., *Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines*, PLDI 2013.

[23] A. Maclaurin et al., *Dex: Array Programming with Typed Indices*, arXiv 2019.